Department of Computer Science

Namal University, Mianwali

FYP Management System

Database Design Document

Course: Database System

Team Members:

Name	Roll Number	Email
Mubashir Hassan	NUM-BSCS-2024-38	bscs24f38@namal.edu.pk
Shumaila Sarfaraz	NUM-BSCS-2024-74	bscs24f74@namal.edu.pk
Sadia	NUM-BSCS-2024-68	bscs24f68@namal.edu.pk

Submission Date: March 12, 2026

Submitted to:
Miss Asiya Batool
Lecturer, Department of Computer Science

REVISION HISTORY

Date	Version	Description
10/06/26	V 2.0	Updated documentation with detailed implementation specifications and verified business rules. Expanded relational schema and functional dependency analysis. Added comprehensive technical details on system architecture, database connectivity, and a complete catalog of database programmability objects including procedures, functions, and triggers.
15/05/26	V 1.0	Initial submission of Database Design Document for FYP Management System. Includes complete data dictionary with data types and constraints, business rules, Entity Relationship Diagram (ERD) with all relationships and cardinalities, multi-value attribute handling through separate tables associative entities for many-to-many relationships, sample data for testing, and test cases validating complete FYP lifecycle from student registration to evaluation results.

Contents

REVISION HISTORY 1

1 PROJECT OVERVIEW 5
- 1.1 INTRODUCTION . . . 5
- 1.2 PROBLEM STATEMENT . . . 6
- 1.3 PROJECT OBJECTIVES . . . 6
  - 1.3.1 General Objective . . . 6
  - 1.3.2 Specific Objectives . . . 6
- 1.4 GITHUB REPOSITORY LINK . . . 7

2 DETAILED DATABASE DESIGN 8
- 2.1 ENTITY . . . 8
- 2.2 DATA DICTIONARY . . . 9
- 2.3 BUSINESS RULES . . . 15

3 LOGICAL DATABASE DESIGN 18
- 3.1 SCHEMA DIAGRAM . . . 18
  - 3.1.1 Schema Description . . . 19
  - 3.1.2 Table Relationships Summary . . . 19
- 3.2 RELATIONAL SCHEMA . . . 20
  - 3.2.1 Core Entities . . . 20
  - 3.2.2 Associative Entities . . . 24
- 3.3 FUNCTIONAL DEPENDENCIES . . . 25
  - 3.3.1 Core Entities Functional Dependencies . . . 25
  - 3.3.2 Associative Entities Functional Dependencies . . . 28
- 3.4 NORMALIZATION . . . 28

4 IMPLEMENTATION 30
- 4.1 LANGUAGE AND FRAMEWORK . . . 30
  - 4.1.1 Backend Framework . . . 30
  - 4.1.2 Frontend Technologies . . . 30
  - 4.1.3 Database System . . . 31
  - 4.1.4 Technology Stack Summary . . . 31
- 4.2 DATABASE CONNECTIVITY . . . 31
  - 4.2.1 Connection Architecture: The Role of the Node.js Backend . . . 31
  - 4.2.2 Database Connection Management: Connection Pooling with mysql2/promise . . . 32
  - 4.2.3 Configuration Parameters . . . 32
  - 4.2.4 Communication Flow . . . 33

4.3 STORED PROCEDURES AND FUNCTIONS 34
4.3.1 Stored Procedures 34
4.3.2 Functions 37
4.3.3 Triggers 38
4.3.4 Views 39
4.4 INTERFACES 39
4.4.1 Login Interface 39
4.4.2 Coordinator Dashboard 39
4.4.3 Evaluator Dashboard 40
4.4.4 System Control Center 40
4.4.5 Student Dashboard 41
4.4.6 Faculty Dashboard 41

REFERENCES **42**

APPENDIX: AI USAGE AND PROMPTS **43**

List of Figures

1.1 Entity Relationship Diagram for FYP Management System 7

3.1 Database Schema Diagram for FYP Management System 18

4.1 Login Interface 39
4.2 Coordinator Dashboard 40
4.3 Evaluator Dashboard 40
4.4 System Control Center 40
4.5 Student Dashboard 41
4.6 Faculty Dashboard 41

Chapter 1

PROJECT OVERVIEW

1.1 INTRODUCTION

The Final Year Project (FYP) is an extremely significant and mandatory component of the undergraduate degree program at Namal University. It allows students to combine practical and theoretical knowledge to solve real-world problems. It is the final major requirement before degree completion, making it important for both students and the university.

Managing the FYP process is not easy at any university. Currently, all FYP-related work is done manually at Namal University. Students submit their project ideas on paper and wait for coordinator approval. There is no proper follow-up system to track which group is at which stage. The university stores FYP data in Excel sheets and other manual tools to track the entire process from group formation to final evaluation. This manual method causes several issues. Data is repeated across multiple files, making it inconsistent and difficult to manage. Records are often lost or misplaced. The entire process is slow and ineffective. As the number of students increases each year, this manual approach becomes harder to handle.

The purpose of the FYP Management System is to digitize and automate all FYP-related activities at Namal University. The system will provide a centralized platform where students can submit their proposals online, coordinators can approve them, and supervisors can monitor progress. The system will track each group from formation to final defense, ensuring no stage is missed.

The system will automate key processes including student project proposal submission, supervisor and co-supervisor assignment, milestone creation, submission uploads, feedback provision, evaluation scheduling, and result recording. It will also maintain complete records of all past and current projects for future reference. It is specifically developed for Namal University, Mianwali, Pakistan. Namal University is a higher education institution that offers undergraduate degree programs in computer science and related fields. The university has a growing number of students each year, making manual FYP management increasingly difficult. The system will operate in a university academic environment. It will be accessed by students, faculty members, department coordinators, and external evaluators from organizations.

1.2 PROBLEM STATEMENT

All tasks associated with Final Year Projects at Namal University are currently handled using manual processes or basic Excel-based programs. The management team, including coordinators and faculty, struggles with numerous challenges due to the rising number of students and projects each year. There is no centralized platform to manage the FYP process from student group formation to final defense, leading to significant issues. Data redundancy occurs as the same information is stored in multiple files and spreadsheets, while data inconsistency exists because different versions of the same data are maintained separately. Loss of records is common as manual documents can be easily misplaced or destroyed. These problems create significant administrative overhead for coordinators and faculty members, making time-consuming processes that delay approvals and feedback. Furthermore, the current system has limited scalability and cannot handle the increasing number of students each year. The absence of proper record management makes it difficult to retrieve past project information for reference and analysis. Students experience delayed proposal approvals, lack of submission transparency, and untimely feedback, which hinders their academic progress. Faculty members bear excessive administrative burden, reducing time available for meaningful student supervision and quality assurance. Coordinators face difficulty accessing submission records, scheduling evaluations, and generating reliable performance reports.

1.3 PROJECT OBJECTIVES

1.3.1 General Objective

To design and implement a fully functional, web-based FYP Management System with a centralized relational database that automates and streamlines the entire FYP lifecycle at Namal University, ensuring efficiency, data integrity, and role-based accessibility for all stakeholders.

1.3.2 Specific Objectives

1. To computerize the entire FYP management process, from student group formation to final assessment, eliminating manual paperwork and reducing processing time by at least 50%.

2. To grant role-specific access to students, supervisors, coordinators, and evaluators, ensuring each user can only view and perform actions relevant to their designated role.

3. To maintain comprehensive and permanent records of all past and present FYPs in a single centralized database, enabling quick retrieval and historical analysis.

4. To remove all forms of data duplication by storing each piece of information only once, ensuring uniformity and consistency across the entire system.

5. To build a scalable system architecture capable of supporting increasing numbers of users, groups, and projects without degradation in performance or response time.

1.4 GITHUB REPOSITORY LINK

LINK: https://github.com/mubashir-hassan-git/FYP-Management-System

Entity Relationship Diagram (ERD):

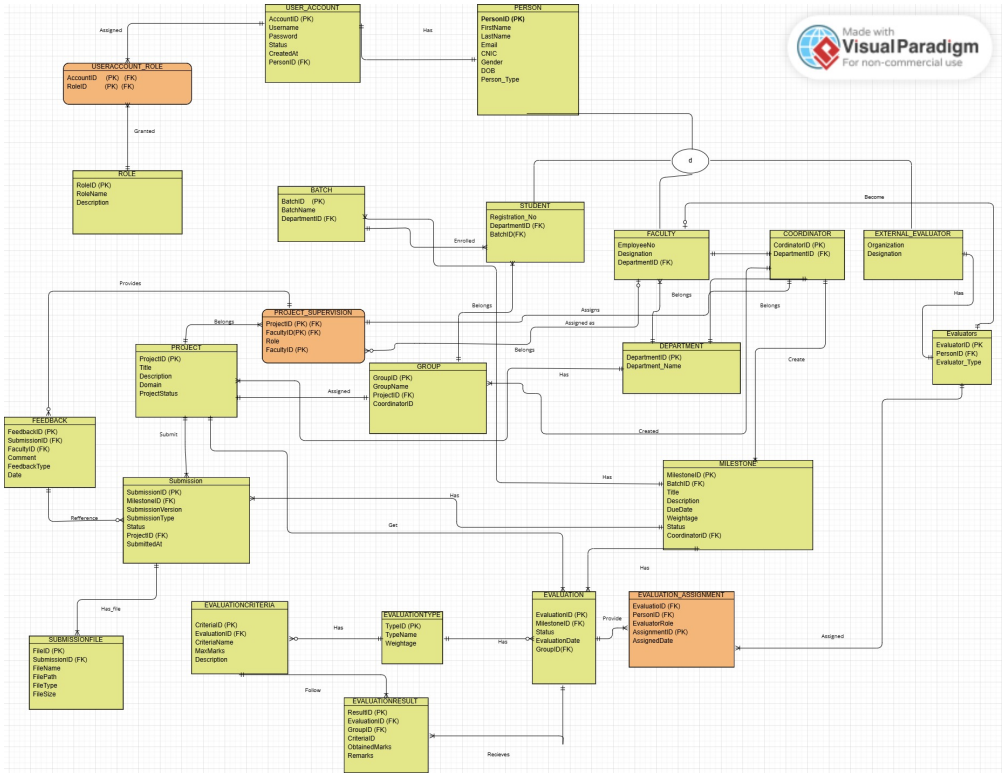

Figure 1.1: Entity Relationship Diagram for FYP Management System

Note: The ERD diagram above illustrates all entities, their attributes, and relationships including cardinalities. The diagram shows primary keys, foreign keys, and the associative entities that resolve many-to-many relationships.

Chapter 2

DETAILED DATABASE DESIGN

2.1 ENTITY

The following entities are central to the FYP Management System database. Each entity represents a real-world object, concept, or event that the system tracks.

1. **PERSON:** A person is an individual who interacts with the system. Persons can be Students, Faculty members or External Evaluators. The PERSON entity serves as the supertype for all user types.
2. **STUDENT:** A student is a person enrolled in a degree program who is required to complete a Final Year Project. Students submit project work, and receive evaluations.
3. **FACULTY:** A faculty member is a person employed by the university who serves as a supervisor or co-supervisor for student projects.
4. **COORDINATOR:** A coordinator is a faculty member appointed to manage FYP activities for a specific department. Coordinators create groups, assign projects, assign supervisors, and create milestones.
5. **EXTERNAL_EVALUATOR:** An external evaluator is a person from industry or another institution who evaluates student projects based on their domain expertise.
6. **EVALUATORS:** An evaluator is a person (internal faculty or external) who is assigned to evaluate student projects. This entity unifies internal and external evaluators.
7. **DEPARTMENT:** A department is an academic unit within the university that offers degree programs and manages FYPs.
8. **BATCH:** A batch represents a group of students admitted in a specific academic session.
9. **GROUP:** A group consists of two students who work together on a single project. Each group is assigned one project and has supervisor and co-supervisor.

10. **PROJECT:** A project is a specific FYP undertaken by a group. It has a title, description, domain, and status.

11. **MILESTONE:** A milestone represents a key deliverable in the FYP lifecycle. Milestones are defined per batch.

12. **SUBMISSION:** A submission is the work uploaded by a student group for a specific milestone.

13. **FEEDBACK:** Feedback is the review provided by a supervisor or co-supervisor on a student submission.

14. **EVALUATION:** An evaluation is an assessment event conducted for a specific milestone and group.

15. **EVALUATION_TYPE:** An evaluation type defines the category of evaluation with its weightage.

16. **EVALUATION_CRITERIA:** Evaluation criteria are the rubrics used to assess a project. Each criterion has a name and maximum marks.

17. **EVALUATION_RESULT:** An evaluation result records the marks obtained by a group for each criterion, along with remarks from evaluators.

18. **EVALUATION_ASSIGNMENT:** An evaluation assignment links evaluators to evaluations, specifying their role.

19. **USER_ACCOUNT:** A user account provides login credentials for a person to access the system.

20. **ROLE:** A role defines the permissions and access level of a user in the system.

21. **SUBMISSION_FILE:** Stores metadata about files uploaded as part of a submission, including file name, path, type, and size.

22. **USERACCOUNT_ROLE:** An associative entity that links user accounts to their assigned roles, allowing for multiple roles per account.

2.2 DATA DICTIONARY

This data dictionary provides a comprehensive overview of the tables and their respective attributes within the database. It details column names, data types, and key constraints, serving as a reference for understanding the database structure.

Table: department

Column Name	Data Type	Constraints	Description
DepartmentID	INT	PK, AUTO_INCREMENT	Unique identifier for department
DepartmentName	VARCHAR(100)	NOT NULL, UNIQUE	Name of department

Table: batch

Column Name	Data Type	Constraints	Description
BatchID	INT	PK, AUTO_INCREMENT	Unique identifier for batch
BatchName	VARCHAR(50)	NOT NULL, UNIQUE	Name of batch (e.g., CS-2021)
DepartmentID	INT	NOT NULL, FK	Dept the batch belongs to

Table: person

Column Name	Data Type	Constraints	Description
PersonID	INT	PK, AUTO_INCREMENT	Unique identifier for person
FirstName	VARCHAR(60)	NOT NULL	First name of person
LastName	VARCHAR(60)		Last name of person
Email	VARCHAR(120)	NOT NULL, UNIQUE	Email address
CNIC	VARCHAR(20)	NOT NULL, UNIQUE	CNIC number
Gender	ENUM	DEFAULT 'Male'	Gender
DOB	DATE		Date of birth
PersonType	ENUM	NOT NULL	Type of person

Table: student

Column Name	Data Type	Constraints	Description
StudentID	INT	PK, FK	References PersonID
RegistrationNo	VARCHAR(30)	NOT NULL, UNIQUE	Registration number
BatchID	INT	NOT NULL, FK	Batch student belongs to
GroupID	INT	FK	Project group

Table: faculty

Column Name	Data Type	Constraints	Description
FacultyID	INT	PK, FK	References PersonID
EmployeeNo	VARCHAR(20)	NOT NULL, UNIQUE	Employee number
Designation	VARCHAR(80)		Job designation
DepartmentID	INT	NOT NULL, FK	Department of faculty

Table: project

Column Name	Data Type	Constraints	Description
ProjectID	INT	PK, AUTO_INCREMENT	Unique identifier for project
Title	VARCHAR(200)	NOT NULL	Title of project
Description	TEXT		Detailed description
Domain	VARCHAR(100)		Project domain
ProjectStatus	ENUM	DEFAULT 'Proposal'	Current status

Table: groups

Column Name	Data Type	Constraints	Description
GroupID	INT	PK, AUTO_INCREMENT	Unique identifier for group
GroupName	VARCHAR(80)	NOT NULL, UNIQUE	Name of group
CoordinatorID	INT	FK	Coordinator for group
ProjectID	INT	FK	Project assigned to group

Table: project_supervision

Column Name	Data Type	Constraints	Description
SupervisionID	INT	PK, AUTO_INCREMENT	Unique identifier
ProjectID	INT	NOT NULL, FK	Project being supervised
FacultyID	INT	NOT NULL, FK	Supervising faculty
Role	ENUM	NOT NULL	Role in supervision

Table: milestone

Column Name	Data Type	Constraints	Description
MilestoneID	INT	PK, AUTO_INCREMENT	Unique identifier
BatchID	INT	NOT NULL, FK	Batch of milestone
Title	VARCHAR(120)	NOT NULL	Milestone title
Description	TEXT		Milestone description
DueDate	DATE	NOT NULL	Due date
Weightage	DECIMAL(5,2)	DEFAULT 0	Weightage in evaluation
Status	ENUM	DEFAULT 'Active'	Current status

Table: submission

Column Name	Data Type	Constraints	Description
SubmissionID	INT	PK, AUTO_INCREMENT	Unique identifier
ProjectID	INT	NOT NULL, FK	Project of submission
MilestoneID	INT	NOT NULL, FK	Milestone for submission
SubmissionVersion	VARCHAR(10)	DEFAULT 'v1.0'	Version number
SubmissionType	ENUM	DEFAULT 'Document'	Type of work
Status	ENUM	DEFAULT 'Submitted'	Status
SubmittedAt	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp

Table: submission_file

Column Name	Data Type	Constraints	Description
FileID	INT	PK, AUTO_INCREMENT	Unique identifier
SubmissionID	INT	NOT NULL, FK	Submission of file
FileName	VARCHAR(200)		Name of file
FilePath	VARCHAR(500)		Storage path
FileType	VARCHAR(80)		MIME type
FileSize	BIGINT		Size in bytes

Table: feedback

Column Name	Data Type	Constraints	Description
FeedbackID	INT	PK, AUTO_INCREMENT	Unique identifier
SubmissionID	INT	NOT NULL, FK	Submission being reviewed
FacultyID	INT	NOT NULL, FK	Faculty giving feedback
Comment	TEXT	NOT NULL	Feedback comment
FeedbackType	ENUM	DEFAULT 'Review'	Type of feedback
FeedbackDate	DATE	DEFAULT (CURRENT_DATE)	Date of feedback

Table: evaluators

Column Name	Data Type	Constraints	Description
EvaluatorID	INT	PK, AUTO_INCREMENT	Unique identifier
PersonID	INT	NOT NULL, UNIQUE, FK	Person who is evaluator
EvaluatorType	ENUM	DEFAULT 'Internal'	Internal or External

Table: evaluation

Column Name	Data Type	Constraints	Description
EvaluationID	INT	PK, AUTO_INCREMENT	Unique identifier
MilestoneID	INT	NOT NULL, FK	Milestone being evaluated
GroupID	INT	NOT NULL, FK	Group being evaluated
EvaluationDate	DATE	NOT NULL	Date of evaluation
Status	ENUM	DEFAULT 'Scheduled'	Status

Table: evaluation_assignment

Column Name	Data Type	Constraints	Description
AssignmentID	INT	PK, AUTO_INCREMENT	Unique identifier
EvaluationID	INT	NOT NULL, FK	Evaluation event
EvaluatorID	INT	NOT NULL, FK	Assigned evaluator
EvaluatorRole	ENUM	DEFAULT 'Internal'	Role of evaluator
AssignedDate	DATE	DEFAULT (CURRENT_DATE)	Date of assignment

Table: evaluation_criteria

Column Name	Data Type	Constraints	Description
CriteriaID	INT	PK, AUTO_INCREMENT	Unique identifier
EvaluationID	INT	NOT NULL, FK	Parent evaluation
CriteriaName	VARCHAR(120)	NOT NULL	Name of criterion
MaxMarks	DECIMAL(6,2)	NOT NULL	Maximum marks
Description	TEXT		Criterion description

Table: evaluation_result

Column Name	Data Type	Constraints	Description
ResultID	INT	PK, AUTO_INCREMENT	Unique identifier
EvaluationID	INT	NOT NULL, FK	Evaluation event
CriteriaID	INT	NOT NULL, FK	Criterion being evaluated
ObtainedMarks	DECIMAL(6,2)	DEFAULT 0	Marks obtained
Remarks	TEXT		Remarks from evaluator

Table: role

Column Name	Data Type	Constraints	Description
RoleID	INT	PK, AUTO_INCREMENT	Unique identifier for role
RoleName	VARCHAR(50)	NOT NULL, UNIQUE	Name of role

Table: user_account

Column Name	Data Type	Constraints	Description
AccountID	INT	PK, AUTO_INCREMENT	Unique identifier
Username	VARCHAR(80)	NOT NULL, UNIQUE	Username for login
Password	VARCHAR(255)	NOT NULL	Hashed password
Status	ENUM	DEFAULT 'Active'	Account status
PersonID	INT	NOT NULL, UNIQUE, FK	Associated person

Table: useraccount_role

Column Name	Data Type	Constraints	Description
ID	INT	PK, AUTO_INCREMENT	Unique identifier
AccountID	INT	NOT NULL, FK	User account
RoleID	INT	NOT NULL, FK	Role assigned

2.3 BUSINESS RULES

The following business rules define how the FYP Management System operates according to the real-world policies and requirements of Namal University. These rules ensure data consistency, integrity, and correct system behavior.

User and Account Management Rules

- **Rule 1:** A person can have only one user account in the system.
- **Rule 2:** A user account must have at least one role assigned to it.
- **Rule 3:** A person can have multiple phone numbers and multiple addresses.
- **Rule 4:** Only faculty members can become coordinators.
- **Rule 5:** A coordinator manages FYP activities for exactly one department.
- **Rule 6:** Each department has exactly one active coordinator at any given time.

Student and Group Management Rules

- **Rule 1:** A student must belong to exactly one batch and one department.
- **Rule 2:** A student must be part of exactly one group.

- **Rule 3:** A group must contain exactly two students.
- **Rule 4:** A group can have only one active project at a time.
- **Rule 5:** A group cannot be formed without coordinator approval.
- **Rule 6:** Only a coordinator can create, approve, or disband a group.

Project and Supervisor Management Rules

- **Rule 1:** A project must be assigned to exactly one group.
- **Rule 2:** A group can work on only one project at a time.
- **Rule 3:** A project must have a supervisor and co-supervisor.
- **Rule 4:** A supervisor can supervise multiple projects simultaneously.
- **Rule 5:** A faculty member can be supervisor for some projects and co-supervisor for others.
- **Rule 6:** A project cannot be assigned to a group without an approved supervisor.
- **Rule 7:** Only a coordinator can assign projects and supervisors to groups.

Milestone and Submission Rules

- **Rule 1:** Milestones are defined per batch.
- **Rule 2:** A milestone must have a due date and weightage before submissions.
- **Rule 3:** A group can submit multiple versions for the same milestone.
- **Rule 4:** A submission must be associated with exactly one project and one milestone.
- **Rule 5:** A submission can have multiple files attached.
- **Rule 6:** Students can only submit work for active milestones.
- **Rule 7:** A supervisor can provide feedback on a submission.

Feedback Management Rules

- **Rule 1:** Only assigned supervisors and co-supervisors can provide feedback.
- **Rule 2:** Feedback can be provided multiple times on the same submission.

Evaluation Management Rules

- **Rule 1:** An evaluation is created for a specific milestone and group.
- **Rule 2:** An evaluation must have at least one evaluation criterion.
- **Rule 3:** Each evaluation criterion has a maximum marks value.
- **Rule 4:** Multiple evaluators can be assigned to the same evaluation.
- **Rule 5:** An evaluator can be assigned to multiple evaluations.

Chapter 3

LOGICAL DATABASE DESIGN

3.1 SCHEMA DIAGRAM

The following schema diagram illustrates the complete database structure of the FYP Management System. The diagram shows all tables, their columns, primary keys, foreign keys, and the relationships between them.

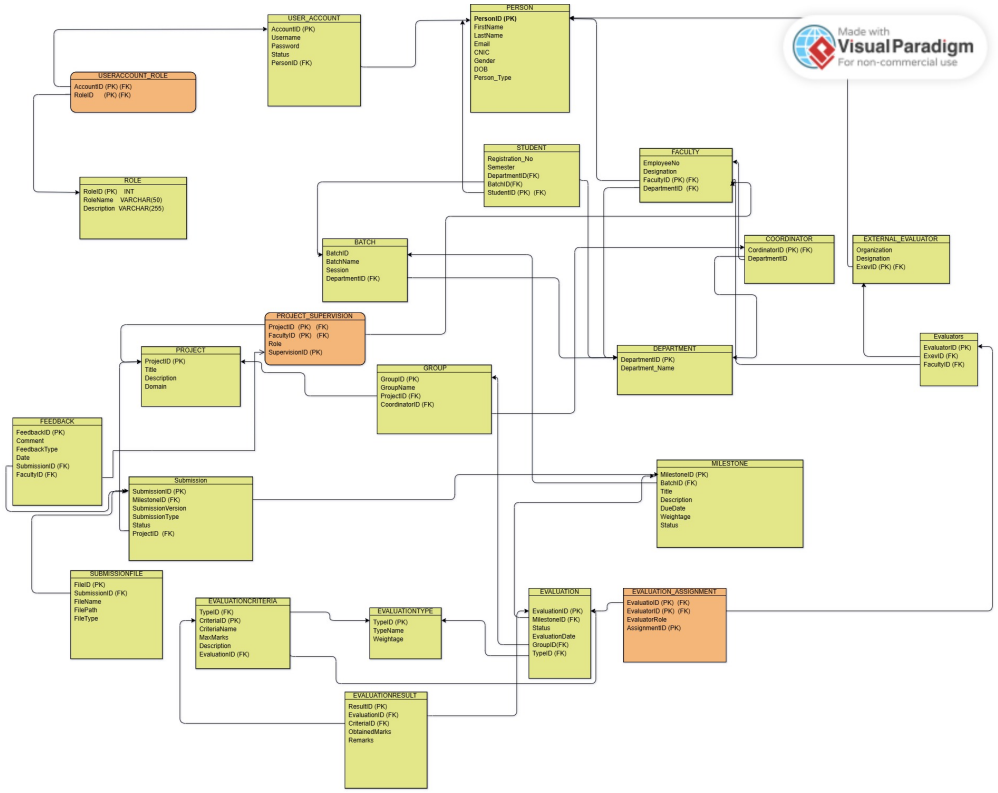

Figure 3.1: Database Schema Diagram for FYP Management System

Note: The schema diagram above presents the complete relational database schema including all 22 tables. Primary keys are underlined, foreign keys are indicated with arrows, and cardinalities are shown at the relationship endpoints. The diagram clearly shows how tables interconnect through foreign key constraints.

3.1.1 Schema Description

The database schema consists of the following key components:

- **Core Tables:** PERSON, USER_ACCOUNT, ROLE, DEPARTMENT, BATCH, STUDENT, FACULTY, PROJECT, GROUP, MILESTONE, SUBMISSION, EVALUATION
- **Supporting Tables:** SUBMISSION_FILE, FEEDBACK, EVALUATORS, EVALUATION_CRITERIA, EVALUATION_RESULT
- **Associative Tables (Junction Tables):** USERACCOUNT_ROLE, PROJECT_SUPERV EVALUATION_ASSIGNMENT

3.1.2 Table Relationships Summary

Table 3.1: Table Relationships Summary

Parent Table	**Child Table**	**Relationship Type**
PERSON	STUDENT	One-to-One
PERSON	FACULTY	One-to-One
PERSON	EVALUATORS	One-to-One
PERSON	USER_ACCOUNT	One-to-One
DEPARTMENT	BATCH	One-to-Many
DEPARTMENT	FACULTY	One-to-Many
BATCH	STUDENT	One-to-Many
BATCH	MILESTONE	One-to-Many
PROJECT	GROUP	One-to-One
PROJECT	SUBMISSION	One-to-Many
PROJECT	PROJECT_SUPERVISION	One-to-Many
GROUP	STUDENT	One-to-Many
GROUP	EVALUATION	One-to-Many
MILESTONE	SUBMISSION	One-to-Many
MILESTONE	EVALUATION	One-to-Many
SUBMISSION	SUBMISSION_FILE	One-to-Many
SUBMISSION	FEEDBACK	One-to-Many
USER_ACCOUNT	USERACCOUNT_ROLE	One-to-Many
ROLE	USERACCOUNT_ROLE	One-to-Many
EVALUATION	EVALUATION_CRITERIA	One-to-Many
EVALUATION	EVALUATION_ASSIGNMENT	One-to-Many
EVALUATION_CRITERIA	EVALUATION_RESULT	One-to-Many
EVALUATORS	EVALUATION_ASSIGNMENT	One-to-Many

3.2 RELATIONAL SCHEMA

3.2.1 Core Entities

1. PERSON

- **Purpose:** Central repository for all individuals in the system (students, faculty, evaluators, coordinators, admins).
- **Primary Key:** PersonID
- **Attributes:** FirstName, LastName, Email, CNIC, Gender, DOB, PersonType
- **Relationships:**
  - student (1:1): StudentID in student references PersonID in person.
  - faculty (1:1): FacultyID in faculty references PersonID in person.
  - evaluators (1:1): PersonID in evaluators references PersonID in person.
  - user_account (1:1): PersonID in user_account references PersonID in person.

2. USER_ACCOUNT

- **Purpose:** Manages user login credentials and account status.
- **Primary Key:** AccountID
- **Attributes:** Username, Password, Status, CreatedAt, LastLogin
- **Foreign Key:** PersonID (references PERSON)

3. ROLE

- **Purpose:** Defines the different roles within the system.
- **Primary Key:** RoleID
- **Attributes:** RoleName, Description

4. DEPARTMENT

- **Purpose:** Stores information about academic departments within the university.
- **Primary Key:** DepartmentID
- **Attributes:** DepartmentName, Code
- **Relationships:**
  - batch (1:N): DepartmentID in batch references DepartmentID in department.
  - faculty (1:N): DepartmentID in faculty references DepartmentID in department.

5. **BATCH**
   - **Purpose:** Manages academic batches, associating them with specific departments.
   - **Primary Key:** BatchID
   - **Attributes:** BatchName, StartYear, EndYear
   - **Foreign Key:** DepartmentID (references DEPARTMENT)
   - **Relationships:**
     - student (1:N): BatchID in student references BatchID in batch.
     - milestone (1:N): BatchID in milestone references BatchID in batch.

6. **STUDENT**
   - **Purpose:** Stores specific details for students.
   - **Primary Key:** StudentID (references PERSON)
   - **Attributes:** RegistrationNo
   - **Foreign Keys:** BatchID (references BATCH), GroupID (references GROUP - nullable)

7. **FACULTY**
   - **Purpose:** Stores specific details for faculty members.
   - **Primary Key:** FacultyID (references PERSON)
   - **Attributes:** EmployeeNo, Designation
   - **Foreign Key:** DepartmentID (references DEPARTMENT)

8. **COORDINATOR (Conceptual Entity)**
   - **Purpose:** Represents individuals designated as coordinators for FYP activities.
   - **Mapping to SQL:** In the physical schema, a coordinator is a PERSON with PersonType = 'Coordinator'. The groups table includes a CoordinatorID (FK to PERSON.PersonID) to link a group to its assigned coordinator.
   - **Attributes (from ERD):** CoordinatorID (PK, FK to PERSON), DepartmentID (FK to DEPARTMENT)

9. EXTERNAL_EVALUATOR (Conceptual Entity)

- **Purpose:** Represents evaluators from outside the university.
- **Mapping to SQL:** In the physical schema, external evaluators are managed within the EVALUATORS table, where EvaluatorType = 'External' and Affiliation stores their organization.
- **Attributes (from ERD):** EvaluatorID (PK, FK to PERSON), Organization, Designation

10. PROJECT

- **Purpose:** Contains information about Final Year Projects.
- **Primary Key:** ProjectID
- **Attributes:** Title, Description, Domain, ProjectStatus
- **Relationships:**
  - groups (1:1): ProjectID in groups references ProjectID in project.
  - submission (1:N): ProjectID in submission references ProjectID in project.

11. GROUP

- **Purpose:** Organizes students into project groups.
- **Primary Key:** GroupID
- **Attributes:** GroupName
- **Foreign Keys:** CoordinatorID (references PERSON), ProjectID (references PROJECT - nullable)

12. MILESTONE

- **Purpose:** Defines project milestones for each batch.
- **Primary Key:** MilestoneID
- **Attributes:** Title, Description, DueDate, Weightage, Status
- **Foreign Key:** BatchID (references BATCH)
- **Relationships:**
  - submission (1:N): MilestoneID in submission references MilestoneID in milestone.
  - evaluation (1:N): MilestoneID in evaluation references MilestoneID in milestone.

13. SUBMISSION

- **Purpose:** Records student submissions for projects against specific milestones.
- **Primary Key:** SubmissionID
- **Attributes:** SubmissionVersion, SubmissionType, Status, SubmittedAt
- **Foreign Keys:** ProjectID (references PROJECT), MilestoneID (references MILESTONE)

14. SUBMISSION_FILE

- **Purpose:** Stores details about files attached to a submission.
- **Primary Key:** FileID
- **Attributes:** FileName, FilePath, FileType, FileSize
- **Foreign Key:** SubmissionID (references SUBMISSION)

15. FEEDBACK

- **Purpose:** Records feedback provided by faculty on submissions.
- **Primary Key:** FeedbackID
- **Attributes:** Comment, FeedbackType, FeedbackDate
- **Foreign Keys:** SubmissionID (references SUBMISSION), FacultyID (references FACULTY)

16. EVALUATORS

- **Purpose:** Stores information about internal and external evaluators.
- **Primary Key:** EvaluatorID
- **Attributes:** EvaluatorType, Affiliation
- **Foreign Key:** PersonID (references PERSON)

17. EVALUATION

- **Purpose:** Manages the evaluation process for groups and milestones.
- **Primary Key:** EvaluationID
- **Attributes:** EvaluationDate, Status
- **Foreign Keys:** MilestoneID (references MILESTONE), GroupID (references GROUP)

18. EVALUATION_CRITERIA

- **Purpose:** Defines the criteria used for a specific evaluation.
- **Primary Key:** CriteriaID
- **Attributes:** CriteriaName, MaxMarks, Description
- **Foreign Key:** EvaluationID (references EVALUATION)

19. EVALUATION_RESULT

- **Purpose:** Records the marks and remarks for each criterion in an evaluation.
- **Primary Key:** ResultID
- **Attributes:** ObtainedMarks, Remarks
- **Foreign Keys:** EvaluationID (references EVALUATION), CriteriaID (references EVALUATION_CRITERIA)

20. EVALUATION_TYPE (Conceptual Entity)

- **Purpose:** Defines types of evaluations with associated weightages.
- **Mapping to SQL:** This conceptual entity is not directly implemented as a separate table in the provided SQL schema.
- **Attributes (from ERD):** TypeID (PK), TypeName, Weightage

3.2.2 Associative Entities

21. USERACCOUNT_ROLE

- **Purpose:** Resolves the many-to-many relationship between USER_ACCOUNT and ROLE.
- **Primary Key:** ID (Surrogate Key)
- **Foreign Keys:** AccountID (references USER_ACCOUNT), RoleID (references ROLE)
- **Unique Constraint:** (AccountID, RoleID)

22. PROJECT_SUPERVISION

- **Purpose:** Resolves the many-to-many relationship between PROJECT and FACULTY.
- **Primary Key:** SupervisionID (Surrogate Key)
- **Attributes:** Role

- **Foreign Keys:** ProjectID (references PROJECT), FacultyID (references FACULTY)
- **Unique Constraint:** (ProjectID, FacultyID)

23. EVALUATION_ASSIGNMENT

- **Purpose:** Resolves the many-to-many relationship between EVALUATION and EVALUATORS.
- **Primary Key:** AssignmentID (Surrogate Key)
- **Attributes:** EvaluatorRole, AssignedDate
- **Foreign Keys:** EvaluationID (references EVALUATION), EvaluatorID (references EVALUATORS)
- **Unique Constraint:** (EvaluationID, EvaluatorID)

3.3 FUNCTIONAL DEPENDENCIES

3.3.1 Core Entities Functional Dependencies

1. PERSON

- **Primary Key:** PersonID
- PersonID → FirstName, LastName, Email, CNIC, Gender, DOB, PersonType
- Email → PersonID, FirstName, LastName, CNIC, Gender, DOB, PersonType
- CNIC → PersonID, FirstName, LastName, Email, Gender, DOB, PersonType

2. USER_ACCOUNT

- **Primary Key:** AccountID
- AccountID → Username, Password, Status, PersonID,
- Username → AccountID, Password, Status, PersonID,
- PersonID → AccountID, Username, Password, Status,

3. ROLE

- **Primary Key:** RoleID
- RoleID → RoleName, Description
- RoleName → RoleID, Description

4. **DEPARTMENT**
   - **Primary Key:** DepartmentID
   - DepartmentID → DepartmentName, Code
   - DepartmentName → DepartmentID, Code
   - Code → DepartmentID, DepartmentName

5. **BATCH**
   - **Primary Key:** BatchID
   - BatchID → BatchName, DepartmentID
   - BatchName → BatchID, DepartmentID
   - DepartmentID, → BatchID, BatchName,

6. **STUDENT**
   - **Primary Key:** StudentID
   - StudentID → RegistrationNo, BatchID, GroupID
   - RegistrationNo → StudentID, BatchID, GroupID

7. **FACULTY**
   - **Primary Key:** FacultyID
   - FacultyID → EmployeeNo, Designation, DepartmentID
   - EmployeeNo → FacultyID, Designation, DepartmentID

8. **PROJECT**
   - **Primary Key:** ProjectID
   - ProjectID → Title, Description, Domain, ProjectStatus

9. **GROUP**
   - **Primary Key:** GroupID
   - GroupID → GroupName, CoordinatorID, ProjectID
   - GroupName → GroupID, CoordinatorID, ProjectID

10. **MILESTONE**
    - **Primary Key:** MilestoneID
    - MilestoneID → BatchID, Title, Description, DueDate, Weightage, Status
    - BatchID, Title → MilestoneID, Description, DueDate, Weightage, Status

11. SUBMISSION

- **Primary Key:** SubmissionID
- SubmissionID → ProjectID, MilestoneID, SubmissionVersion, SubmissionType, Status,
- ProjectID, MilestoneID, SubmissionVersion → SubmissionID, SubmissionType, Status, S

12. SUBMISSION_FILE

- **Primary Key:** FileID
- FileID → SubmissionID, FileName, FilePath, FileType, FileSize

13. FEEDBACK

- **Primary Key:** FeedbackID
- FeedbackID → SubmissionID, FacultyID, Comment, FeedbackType, FeedbackDate

14. EVALUATORS

- **Primary Key:** EvaluatorID
- EvaluatorID → PersonID, EvaluatorType, Affiliation
- PersonID → EvaluatorID, EvaluatorType, Affiliation

15. EVALUATION

- **Primary Key:** EvaluationID
- EvaluationID → MilestoneID, GroupID, EvaluationDate, Status
- MilestoneID, GroupID → EvaluationID, EvaluationDate, Status

16. EVALUATION_CRITERIA

- **Primary Key:** CriteriaID
- CriteriaID → EvaluationID, CriteriaName, MaxMarks, Description
- EvaluationID, CriteriaName → CriteriaID, MaxMarks, Description

17. EVALUATION_RESULT

- **Primary Key:** ResultID
- ResultID → EvaluationID, CriteriaID, ObtainedMarks, Remarks
- EvaluationID, CriteriaID → ResultID, ObtainedMarks, Remarks

3.3.2 Associative Entities Functional Dependencies

18. USERACCOUNT_ROLE

- ID → AccountID, RoleID
- AccountID, RoleID → ID

19. PROJECT_SUPERVISION

- SupervisionID → ProjectID, FacultyID, Role
- ProjectID, FacultyID → SupervisionID, Role

20. EVALUATION_ASSIGNMENT

- AssignmentID → EvaluationID, EvaluatorID, EvaluatorRole, AssignedDate
- EvaluationID, EvaluatorID → AssignmentID, EvaluatorRole, AssignedDate

3.4 NORMALIZATION

All tables have been normalized to Third Normal Form (3NF). Below is the complete list of 3NF relations:

1. DEPARTMENT (DepartmentID, DepartmentName, Code)
2. BATCH (BatchID, BatchName, StartYear, EndYear, DepartmentID)
3. PERSON (PersonID, FirstName, LastName, Email, CNIC, Gender, DOB, PersonType)
4. STUDENT (StudentID, RegistrationNo, BatchID, GroupID)
5. FACULTY (FacultyID, EmployeeNo, Designation, DepartmentID)
6. PROJECT (ProjectID, Title, Description, Domain, ProjectStatus)
7. GROUP (GroupID, GroupName, CoordinatorID, ProjectID)
8. MILESTONE (MilestoneID, BatchID, Title, Description, DueDate, Weightage, Status)
9. SUBMISSION (SubmissionID, MilestoneID, ProjectID, SubmissionVersion, SubmissionType, Status, SubmittedAt)
10. SUBMISSION_FILE (FileID, SubmissionID, FileName, FilePath, FileType, FileSize)
11. FEEDBACK (FeedbackID, SubmissionID, FacultyID, Comment, FeedbackType, FeedbackDate)
12. EVALUATION (EvaluationID, MilestoneID, GroupID, EvaluationDate, Status)

13. EVALUATION_CRITERIA (CriteriaID, EvaluationID, CriteriaName, MaxMarks, Description)
14. EVALUATION_RESULT (ResultID, EvaluationID, CriteriaID, ObtainedMarks, Remarks)
15. ROLE (RoleID, RoleName, Description)
16. USER_ACCOUNT (AccountID, Username, Password, Status, PersonID, CreatedAt, LastLogin)
17. USERACCOUNT_ROLE (AccountID, RoleID)
18. PROJECT_SUPERVISION (ProjectID, FacultyID, Role)
19. EVALUATION_ASSIGNMENT (EvaluationID, EvaluatorID, EvaluatorRole, AssignedDate)
20. EVALUATORS (EvaluatorID, PersonID, EvaluatorType, Affiliation)

Chapter 4

IMPLEMENTATION

4.1 LANGUAGE AND FRAMEWORK

4.1.1 Backend Framework

- The backend is developed using **Node.js** and **Express.js**.
- Express.js is used to build RESTful APIs.
- It provides efficient routing and middleware support.
- The framework supports role-based access control.
- The backend uses **CommonJS modules**.
- **jsonwebtoken (JWT)** is used for authentication and authorization.
- **multer** is used for file upload handling.

4.1.2 Frontend Technologies

- The frontend is developed using **HTML5**, **Vanilla JavaScript**, and **Tailwind CSS**.
- HTML5 provides the structure of the web application.
- JavaScript handles client-side functionality and interactivity.
- Tailwind CSS is used for responsive and consistent user interface design.
- Tailwind CSS is integrated through a CDN.
- The interface supports multiple dashboards including Student Dashboard, Faculty Dashboard, Coordinator Dashboard, Evaluator Dashboard, and Admin Dashboard.
- **Google Fonts** are used to improve typography.
- **Material Symbols** are used for icons and better usability.

4.1.3 Database System

- **MySQL** is used as the Relational Database Management System (RDBMS).
- MySQL provides reliable data storage and management.
- The database uses Stored Procedures, Triggers, and Functions.
- These features help enforce business rules directly at the database level.
- MySQL ensures secure and efficient handling of FYP-related data.

4.1.4 Technology Stack Summary

Table 4.1: Technology Stack for FYP Management System

Category	**Technology**	**Purpose**
Backend Framework	Node.js, Express.js	RESTful APIs, routing, middleware
Frontend Technologies	HTML5, Vanilla JS, Tailwind CSS	Structure, interactivity, UI
Database System	MySQL	Relational DB with procedures
Authentication	JSON Web Token (JWT)	Auth & authorization
File Upload	multer	File handling
UI Enhancements	Google Fonts, Material Symbols	Typography & icons

4.2 DATABASE CONNECTIVITY

The Graphical User Interface (GUI) application of the Namal FYP Portal establishes communication with the MySQL database through a robust, three-tier architecture. This architecture comprises the client-side GUI, an intermediary Node.js backend server, and the MySQL relational database management system. This design pattern ensures a clear separation of concerns, enhanced security, and improved scalability for the application.

4.2.1 Connection Architecture: The Role of the Node.js Backend

The GUI application, primarily composed of HTML, CSS, and JavaScript, does not directly interact with the database. Instead, all database operations are routed through the Node.js backend server, which functions as a dedicated API layer. This design choice offers several advantages:

- **Security:** Prevents direct exposure of database credentials and schema to the client-side, mitigating risks such as SQL injection and unauthorized data access.

- **Business Logic Encapsulation:** Centralizes complex business rules and data validation on the server, ensuring consistency regardless of the client application.
- **Performance Optimization:** Allows the backend to manage database connections efficiently, process requests, and perform server-side caching or data aggregation.
- **Scalability:** The backend can be scaled independently of the frontend and database, enabling the system to handle increased user loads effectively.

4.2.2 Database Connection Management: Connection Pooling with mysql2/promise

Database connectivity between the Node.js backend and the MySQL server is managed using the `mysql2/promise` library. This library provides an asynchronous, promise-based interface for interacting with MySQL, which is highly suitable for Node.js's non-blocking I/O model. A critical component of this management is the implementation of a Connection Pool.

Instead of initiating a new database connection for every incoming client request, which is resource-intensive and can lead to performance bottlenecks, the system maintains a pool of preestablished, reusable connections. When the backend needs to perform a database operation, it requests a connection from this pool. Once the operation is complete, the connection is returned to the pool, ready for the next request. This approach offers significant benefits:

- **Reduced Overhead:** Eliminates the computational cost and latency associated with repeatedly opening and closing connections.
- **Improved Performance:** Faster response times due to readily available connections.
- **Enhanced Scalability:** Efficiently handles a large number of concurrent requests by sharing a limited set of connections.
- **Resource Management:** Prevents the database server from being overwhelmed by an excessive number of simultaneous connection attempts.

4.2.3 Configuration Parameters

The database connection pool is meticulously configured within the `src/config/db.js` file. The parameters are critical for defining how the application connects to and interacts with the MySQL database:

Table 4.2: Database Connection Pool Configuration

Parameter	Value	Description
host	localhost	DB server hostname
port	3306	MySQL default port
user	root	Database username
password	123456789	Database password
database	fyb_db	Database name
waitForConnections	true	Queue when busy
connectionLimit	10	Max concurrent connections
queueLimit	0	Unlimited queue size

4.2.4 Communication Flow

The interaction between the GUI, backend, and database follows a well-defined sequence:

1. **User Interaction (GUI):** A user initiates an action on the frontend (e.g., clicking a button, submitting a form). This action triggers a JavaScript function in the client-side application.

2. **Frontend Request (HTTP/HTTPS):** The JavaScript function constructs an asynchronous HTTP or HTTPS request (e.g., using the `fetch` API or `XMLHttpRequest`) and sends it to a specific endpoint on the Node.js Express.js server. This request typically carries data in JSON format.

3. **Backend Processing (Express.js):** The Express.js server receives the HTTP request. Its routing mechanism directs the request to the appropriate controller function based on the URL path and HTTP method. Authentication and authorization middleware are applied to validate the user's identity and permissions.

4. **Database Connection Acquisition:** The controller function, requiring database access, requests a connection from the `mysql2` connection pool. If a connection is available, it is immediately provided; otherwise, the request waits in the queue until one becomes free.

5. **Database Query Execution (TCP/IP):** Using the acquired connection, the controller executes the necessary SQL queries or invokes stored procedures (e.g., `sp_register_student`, `sp_get_student_profile`) against the MySQL database. This communication occurs over TCP/IP protocol.

6. **Database Response:** The MySQL database processes the query, performs the requested operation, and returns the results (e.g., data rows, status codes, error messages) back to the Node.js backend via the established connection.

7. **Backend Response (HTTP/HTTPS):** The Node.js backend processes the database results, formats them (typically as JSON), and sends an HTTP/HTTPS response back to the originating frontend client.

8. **Frontend Rendering:** The frontend client receives the response and updates the GUI accordingly, displaying new data, confirmation messages, or error notifications to the user.

This structured communication flow ensures efficient, secure, and reliable data exchange throughout the Namal FYP Portal system.

4.3 STORED PROCEDURES AND FUNCTIONS

4.3.1 Stored Procedures

sp_register_student

Object Name	sp_register_student
Object Type	Stored Procedure
Description	Registers a new student and creates records in Person, Student, UserAccount, and UserRole tables within a transaction.
Input Parameters	firstName, lastName, email, cnic, gender, dob, registrationNo, batchId, password
Output	StudentID (PersonID)

sp_register_faculty

Object Name	sp_register_faculty
Object Type	Stored Procedure
Description	Registers a faculty member and creates associated records.
Input Parameters	firstName, lastName, email, cnic, gender, dob, employeeNo, designation, deptId, password
Output	FacultyID (PersonID)

sp_create_group

Object Name	sp_create_group
Object Type	Stored Procedure
Description	Creates a new FYP group and assigns two eligible students.
Input Parameters	groupName, coordinatorId, studentId1, studentId2
Output	GroupID

sp_assign_supervisors

Object Name	sp_assign_supervisors
Object Type	Stored Procedure
Description	Assigns supervisor and optional co-supervisor to a project.
Input Parameters	projectId, supervisorId, coSupervisorId
Output	Confirmation Message

sp_record_marks

Object Name	sp_record_marks
Object Type	Stored Procedure
Description	Inserts or updates evaluation marks while validating maximum marks.
Input Parameters	evaluationId, criteriaId, obtained, remarks
Output	None

sp_get_student_profile

Object Name	sp_get_student_profile
Object Type	Stored Procedure
Description	Retrieves complete student profile.
Input Parameters	studentId
Output	Student Profile Result Set

sp_get_faculty_profile

Object Name	sp_get_faculty_profile
Object Type	Stored Procedure
Description	Retrieves complete faculty profile.
Input Parameters	facultyId
Output	Faculty Profile Result Set

sp_update_profile

Object Name	sp_update_profile
Object Type	Stored Procedure
Description	Updates editable profile information.
Input Parameters	personId, firstName, lastName, gender, dob
Output	None

sp_change_password

Object Name	sp_change_password
Object Type	Stored Procedure
Description	Changes account password.
Input Parameters	personId, newPassHash
Output	None

sp_toggle_account_status

Object Name	sp_toggle_account_status
Object Type	Stored Procedure
Description	Activates or deactivates a user account and records audit information.
Input Parameters	personId, status
Output	None

sp_system_stats

Object Name	sp_system_stats
Object Type	Stored Procedure
Description	Retrieves overall system statistics for dashboard.
Input Parameters	None
Output	Statistics Result Set

4.3.2 Functions

fn_full_name

Object Name	fn_full_name
Object Type	Function
Description	Returns full name of a person.
Input Parameters	personId INT
Output	VARCHAR(125)

fn_eval_percentage

Object Name	fn_eval_percentage
Object Type	Function
Description	Calculates evaluation percentage.
Input Parameters	evaluationId INT
Output	DECIMAL(5,2)

fn_submission_count

Object Name	fn_submission_count
Object Type	Function
Description	Counts project submissions.
Input Parameters	projectId INT
Output	INT

fn_group_progress

Object Name	fn_group_progress
Object Type	Function
Description	Calculates milestone completion percentage.
Input Parameters	groupId INT
Output	DECIMAL(5,2)

4.3.3 Triggers

trg_before_insert_submission

Object Name	trg_before_insert_submission
Object Type	Trigger
Description	Validates milestone status and generates submission metadata.
Event	BEFORE INSERT ON Submission
Action Performed	Updates SubmissionVersion and Status fields before insert.

trg_after_update_user_status

Object Name	trg_after_update_user_status
Object Type	Trigger
Description	Tracks account status changes.
Event	AFTER UPDATE ON UserAccount
Action Performed	Inserts record into AuditLog.

trg_after_delete_person

Object Name	trg_after_delete_person
Object Type	Trigger
Description	Tracks deletion activities.
Event	AFTER DELETE ON Person
Action Performed	Inserts deletion record into AuditLog.

4.3.4 Views

view_student_milestone_dashboard

Object Name	view_student_milestone_dashboard
Object Type	View
Description	Provides a consolidated dashboard for student groups.
Information Displayed	Group details, project title, milestone status, deadlines, submissions, and progress tracking.

4.4 INTERFACES

This section presents the graphical user interfaces developed for the FYP Management System. Screenshots of major interfaces are included to demonstrate the functionalities provided to different users of the system.

4.4.1 Login Interface

The login interface provides secure access to the system by authenticating users based on their username and password. It serves as the entry point for students, faculty members, coordinators, evaluators, and administrators.

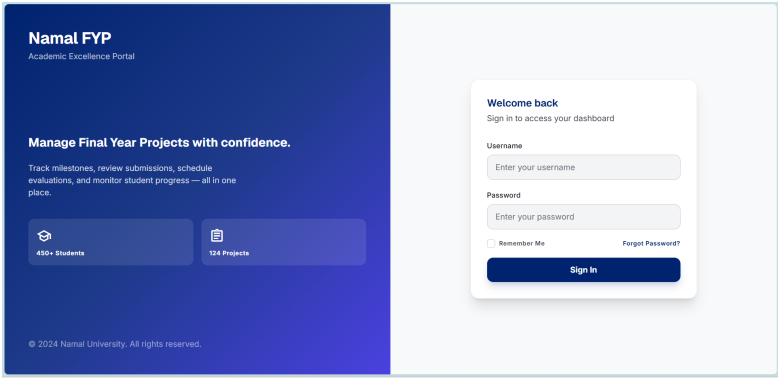

Figure 4.1: Login Interface

4.4.2 Coordinator Dashboard

The Coordinator Dashboard enables the coordinator to manage students, faculty members, groups, projects, supervision assignments, milestones, submissions, and evaluations. It acts as the central control area for managing the overall FYP process.

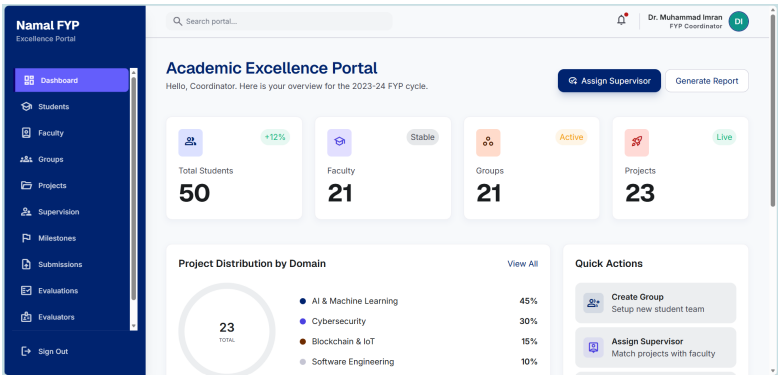

Figure 4.2: Coordinator Dashboard

4.4.3 Evaluator Dashboard

The Evaluator Dashboard provides access to assigned evaluations, rubrics, marking sheets, evaluation results, and evaluator profile management, ensuring an organized and efficient assessment process.

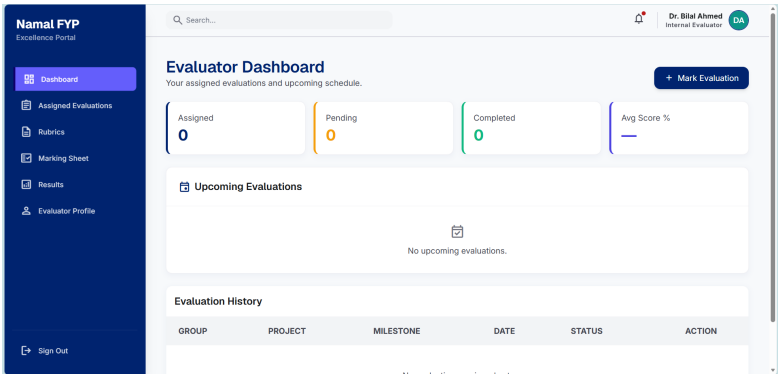

Figure 4.3: Evaluator Dashboard

4.4.4 System Control Center

The System Control Center provides administrative functionalities including user account management, access control, and administrator profile management.

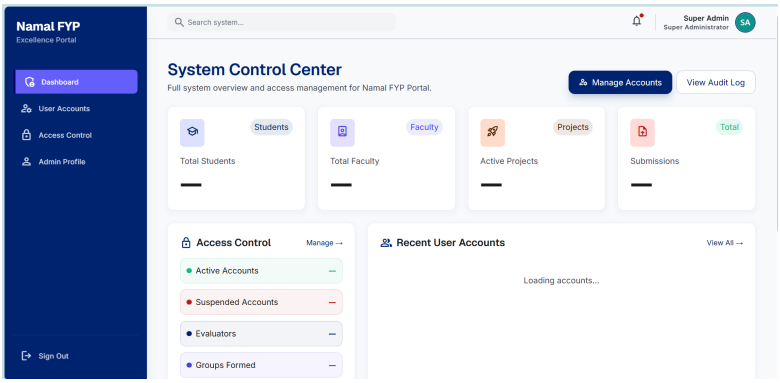

Figure 4.4: System Control Center

4.4.5 Student Dashboard

The Student Dashboard provides students with access to project details, group members, milestone submissions, feedback, evaluation information, and profile management.

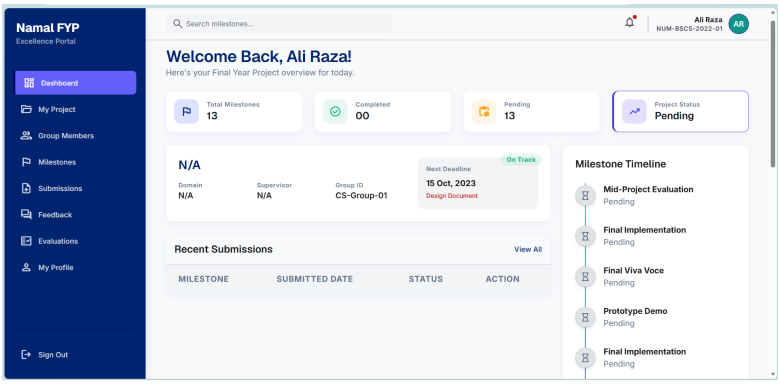

Figure 4.5: Student Dashboard

4.4.6 Faculty Dashboard

The Faculty Dashboard presents an overview of faculty activities and enables faculty members to supervise projects, review submissions, provide feedback, conduct evaluations, and manage their profiles.

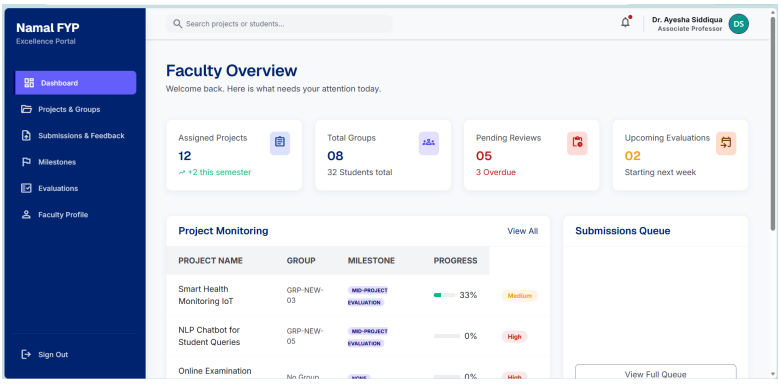

Figure 4.6: Faculty Dashboard

REFERENCES

1. M. Hassan, S. Sarfaraz, and Sadia, "FYP Management System - Software Requirements Specification," GitHub Repository, 2025.
2. A. Mateen, "University Final Year Project Management System," GitHub Repository, 2023.
3. OpenAI, "ChatGPT - Large Language Model," 2024.
4. Deepseek, "Deepseek - Large Language Model," 2024.
5. Visual Paradigm, "ERD Tools and Database Design," 2024.

APPENDIX: AI USAGE AND PROMPTS

This section documents the use of Artificial Intelligence (AI) tools in the development of this Database Design Document.

1. **ChatGPT:** Create complete data dictionary with appropriate data types
2. **ChatGPT:** Identify and list all business rules for FYP system
3. **ChatGPT:** List all entities with clear descriptions
4. **Claude:** Rewrite problem statement as professional paragraph
5. **ChatGPT:** Generate sample data rows for all tables
6. **Chatgpt:** Verify foreign key relationships and cardinalities
7. **ChatGPT:** Write concise introduction paragraph
8. **ChatGPT:** Format stored procedures into professional tables
9. **ChatGPT:** Create technology stack table
10. **Deepseek:** Explain connection pooling mechanism
11. **ChatGPT:** Generate LaTeX code with proper table formatting
12. **ChatGPT:** Design ERD diagram for FYP Management System
13. **Deepseek:** Normalize database tables to 3NF
14. **ChatGPT:** Write functional dependencies for all entities
15. **Deepseek:** Create relational schema from ERD
16. **ChatGPT:** Generate SQL scripts for table creation
17. **ChatGpt:** Design stored procedures for student registration
18. **ChatGPT:** Create trigger for submission validation
19. **Deepseek:** Write function to calculate evaluation percentage
20. **ChatGPT:** Design dashboard interfaces for different roles
21. **Deepseek:** Implement JWT authentication flow

22. **ChatGPT:** Create connection pool configuration
23. **Deepseek:** Design evaluation marking rubric system
24. **ChatGPT:** Generate milestone submission workflow
25. **Deepseek:** Create feedback management system
26. **ChatGPT:** Design group formation business rules
27. **Deepseek:** Implement supervisor assignment logic
28. **Gemini:** Create view for student milestone dashboard
29. **Claude:** Design audit logging system
30. **ChatGPT:** Generate sample test data for all tables
31. **ChatGPT:** Write documentation for API endpoints